BAT
Software Development Lifecycle Playbook

SDLC Playbook Change Log.

Version	Date	Section	Change Description	Author	Reviewer	Approval Status
0.1	30/12/2021	First draft	• Initial creation of the SDLC playbook	R. Pellegrini	Najeeb Younossi	Approved
0.2	26/01/2022	Sections 4,7,10	• Updated practices for SDLC	R. Pellegrini	Najeeb Younossi	Approved
0.3	05/10/2023		• Updates to Branching Strategy	Najeeb Younossi	Najeeb Younossi	Approved
0.4	15/01/2025	Sections 1,4, 5, 9,11 addition of section 8,14	• Consolidation of Release management process • Updates to all Sections 1,4, 5, 9 addition of section 8.14	Ahsan Tahir	Kevin Yu/ Najeeb Younossi	Approved
0.5	03/03/2025	Section 5 Section 14	• Changes to the Versioning 5.8 • Fully defined Software Test Strategy	Ahsan Tahir Chris Lu	Kevin Yu	Approved
0.6	09/09/2025	Section 5.1	• Changes to the repo creation process	Dennis Du Ahsan Tahir	Kevin Yu	Approved
0.7	17/09/2025	Section 5.2	• Build Pipeline Process	Dennis Du Ahsan Tahir	Kevin Yu	Approved
0.8	02/12/2025	Section 1.3/14.3	• Update the content of FW stability test	Chris Lu	Kevin Yu	Approved
	24/08/2026	Section 8	• Add Development Env Standard	Kent Lee		
	24/08/2026	Section 1.3 / 7 / 9 / 15.3	• Update all reference doc link due to storage link changed	Kevin Yu		
0.9	22/10/2026	Section 3.3	• Update to the Sprint Planning	Wency Xie	Ahsan Tahir & Kevin Yu	Approved

		Section 5.7 Section 6 Section 15.3 & 15.5.3	• Update to the Branching Policy • Update to the Pull Requests • Update to the Software Testing • Update to the Software test bug lifecycle			

Table of Contents

1 INTRODUCTION **5**

1.1 SCOPE 5
1.2 GLOSSARY 5
1.3 REFERENCES 5

2 SUMMARY **6**

3 AGILE PRINCIPLES **7**

3.1 BACKLOG STRUCTURE 7
3.1.1 USER STORIES FORMAT 7
3.1.2 DEFINITION OF READY 7
3.1.3 DEFINITION OF DONE 7
3.2 BACKLOG REFINEMENT 8
3.3 SPRINT PLANNING 8
3.3.1 EFFORT SIZING 8
3.4 STAND-UPS 8
3.5 RETROSPECTIVE 9

4 DEVOPS - CONTINUOUS INTEGRATION & CONTINUOUS DELIVERY **10**

4.1 DEV BRANCH 10
4.2 QA & PROD BRANCHES 10

5 SOURCE CODE MANAGEMENT **12**

5.1 CREATION OF A NEW REPOSITORY 12
5.2 CREATION OF A NEW PIPELINE 13
5.3 BRANCH STRATEGY 14
5.4 HOT FIX MANAGEMENT 15
5.5 FEATURE BRANCH NAMING 16
5.6 BRANCHES NAMING CONVENTIONS 16
5.7 BRANCH POLICIES 16
5.8 COMMIT BEST PRACTICES 17
5.9 SEMANTIC VERSIONING STRUCTURE 18

6 PULL REQUESTS 19

6.1 GUIDELINES ON PRS 19
6.2 PR TEMPLATE 20
I. DESCRIPTION 20
II. STEPS TO REPRODUCE BUG AND VALIDATE SOLUTION 20
III. PR CHECKLIST 20
IV. BREAKING CHANGE 20
V. TESTING 20
VI. DEPENDENCIES 21
6.3 CODE REVIEWS 21

7 CODING STANDARDS 23

8 DEVELOPMENT ENVIRONMENT STANDARDS 24

9 CODE-RELATED SECURITY PRACTICES 25

10 PROCESS RELATED SECURITY PRACTICES 26

10.1 SECRETS MANAGEMENT 26
10.1.1 GENERAL GUIDELINES 26
10.1.2 MANAGING SECRETS IN ADO ON A PER-BRANCH BASIS 26
10.1.3 CREDENTIALS SCANNING 27

11 SDLC RACI 28

12 RELEASE CADENCE 29

13 STORAGE OF SECRETS AND SIGNING 31

13.1 SAFE SECRETS STORAGE 31
13.2 GENERAL GUIDELINES 31
13.3 MANAGING SECRETS IN ADO ON A PER-BRANCH BASIS 31
13.4 SECURE FILES 32
13.4.1 ADD A SECURE FILE 32
13.4.2 SIGNING AND PUBLISHING 33

14 RELEASE NOTES 34

14.1 BASIC RELEASE NOTES TEMPLATE 34
14.2 SOFTWARE RELEASE INFORMATION 34
14.3 USER STORIES 34
14.4 OVERVIEW 34
14.5 FEATURES 34
14.6 BUG FIX AND CODE REVIEW 34
14.7 GENERIC INFORMATION 34

15 SOFTWARE TEST STRATEGY 35

15.1 SOFTWARE TEST INPUT AND TEST SCOPE 35
15.1.1 SOFTWARE TEST INPUT 35
15.1.2 SOFTWARE TEST SCOPE 35

15.2 SOFTWARE TEST ENVIRONMENT, TOOLS AND METHODOLOGY 36
15.2.1 SOFTWARE TEST ENVIRONMENT 36
15.2.2 SOFTWARE TEST CONSUMABLE 36
15.2.3 SOFTWARE TEST TOOLS REQUIREMENT 36
15.2.4 SOFTWARE TEST SAMPLE QUANTITY REQUIREMENT 37
15.3 SOFTWARE TEST TYPE 37
15.4 SOFTWARE TEST PLAN MANAGEMENT 38
15.4.1 SOFTWARE TEST PLAN 38
15.4.2 SOFTWARE TEST CASE 39
15.5 SOFTWARE TEST BUG MANAGEMENT 40
15.5.1 SOFTWARE TEST BUG OVERVIEW 40
15.5.2 SOFTWARE TEST BUG ASSESSMENT 41
15.5.3 SOFTWARE TEST BUG LIFECYCLE 42
15.5.4 SOFTWARE TEST BUG CREATION/RESOLVE REQUIREMENT 46
15.6 SOFTWARE TEST REPORT MANAGEMENT 48
16 FW PRODUCTION RELEASE PROCESS 49
16.1 DEVELOPMENT PHASE 49
16.1.1 DEVELOPMENT 49
16.1.2 BUILD 49
16.1.3 UNIT TEST 49
16.1.4 SAST (STATIC APPLICATION SECURITY TESTING) 50
16.1.5 DEV RELEASE 50
16.2 QA PHASE 50
16.2.1 BUILD 50
16.2.2 MANUAL & AUTOMATED FUNCTIONAL TEST 50
16.2.3 REGRESSION TEST 50
16.2.4 SPRINT RELEASE 50
16.2.5 QA RELEASE THIS IS SOFTWARE THAT IS IN A BETA STATUS AND SHOULD BE TESTED INTERNALLY BY BAT. 50
16.3 PRODUCTION/MAIN RELEASE PHASE 50
16.3.1 BUILD 50
16.3.2 AUTOMATED TEST 50
16.3.3 FULL FUNCTION TEST 50
16.3.4 STABILITY TEST 51
16.3.5 PRODUCTION VERIFICATION 51
16.3.6 PRD RELEASE THIS IS SOFTWARE THAT IS READY TO BE RELEASED TO THE END CONSUMER AND FLASHED ONTO PRODUCTION DEVICES. 51

1 INTRODUCTION

1.1 SCOPE

This document provides a playbook for the Development Process within the Software Development Life Cycle (SDLC) paradigm to be applied to the software design.

The playbook collects the guidelines and practices recommended for the main Development activities, with the goal of fostering code quality and security through an emphasis on Continuous Integration as a consistent practice to ensure code and application quality via automated pipelines.

Add the intended users/audience

1.2 GLOSSARY

- **BFP(BAT Firmware Platform**: the new generation Embedded Software Platform for the BAT non-combustion connected products
- **ADO**: Azure DevOps
- **CI**: Continuous Integration
- **CD**: Continuous Delivery
- **SDLC**: Software Development Life Cycle
- **SAST**: Static Analysis Software Test
- **PR**: Pull Request
- **QA**: Quality Assurance
- **PO**: Product Owner

Commented [AT(1]: Replace with SSDLC

1.3 REFERENCES

1. https://docs.microsoft.com/en-us/azure/devops/boards/?view=azure-devops
2. https://github.com/microsoft/code-with-engineering-playbook/
3. https://trunkbaseddevelopment.com/
4. BAT_IOT_SSDLC_Nov22_AL
5. BAT FW Coding Standard
6. FW Stability Test Guideline

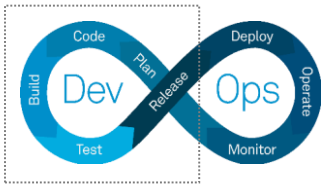
Commented [AT2]: Suggest add SonarQube Wiki link: https://dev.azure.com/ado-bat-uksouth-prod-01/FW_Platform/_wiki/wikis/FW_Platform.wiki/1721/SonarQube-%F0%9F%94%8D

2 SUMMARY

This document provides a playbook for the Development Process within the Software Development Life Cycle (SDLC) paradigm to be applied to the BFP (BAT Firmware Platform) program.

The playbook collects the guidelines and practices recommended for the main Development activities, with the goal of fostering code quality and security through an emphasis on Continuous Integration as a consistent practice to ensure code and application quality via automated pipelines.

It is to be noted that the current edition of this playbook focuses on the Development process activities; a coverage of the Operations process activities is left for a future update.



3 AGILE PRINCIPLES

The development process will employ a Scrum Agile methodology; however, it is recognized that the BAT Product Development paradigm is still structurally of a waterfall type, therefore leading practically to a “Scrumfall” approach.

The following sections are not meant to provide a primer on the Scrum Agile paradigm and ceremonies, which are assumed to be known to the Project Team, but to summarize the key highlights and recommendations for the main activities, as applicable to the Dev process.

3.1 Backlog structure

The backlog items shall be created and organized in ADO, following a hierarchical structure of Epics, Features, User Stories, Tasks.
It is recommended to make use of the ADO facilities to link backlog items according to their hierarchy (parent – child), so as to facilitate the organization of stories under logically related areas.

3.1.1 User stories format

User stories should be formulated in the format:

“As <who> I want <what> so that <why>”.

While this is well understood for functional requirements, we recommend to use this formulation also for non-functional requirements, since it forces:

1. to extract latent functional requirements which are “hidden” inside non-functional statements
2. to focus on what is the benefit of a non-functional requirement and who effectively benefits it: this avoids that non-functional requirements are over-engineered or under-engineered due to being left to the interpretation and sensitivity of the developer on the specific theme.

As regards Acceptance Criteria, it is suggested to use a formulation of the type:

“Given... (the context)”, “when... (the action or event) “, “then... (the observable outcome)”

This pattern forces to capture what is the outcome that the implementation shall provide based on a context and as a result of a certain event or action.

3.1.2 Definition of Ready

The Definition of Ready is a set of criteria by which a Story is considered ready for being part of a sprint planning and qualified with the necessary details for the team to understand the work to be done.

Definition of Ready criteria should comprise the following checklist items:

- does the description have the details including any input values required to implement the story?
- does the story have clear and complete acceptance criteria?
- is the user story un-blocked from any dependencies (e.g. deliverables from other teams, other work items to be completed)

3.1.3 Definition of Done

The Definition of Done criteria for a story should comprise at least the following items:

1. Acceptance criteria are satisfied
2. Code builds with no error

Commented [AT3]: Should this be our DoD?
Development Completed
Unit Tested
Static Code Analysis
PR (code review)
QA Validation Completed

Commented [AT4]: Do we need this to be part of our DoD

3. Unit tests developed and passed
4. Integration tests developed and passed
5. Code review is complete
6. The feature is merged into the relevant branch

It is to be noted that most of the above items will be organized in automated CI/CD pipelines that will be triggered by PRs.

Commented [AT5]: What is the Integration test? Who performs the IT, Developer/QA?

Commented [AT6]: Suggest add:
CI/CD validation checks, including applicable SonarQube quality gates.

3.2 Backlog Refinement

Backlog Refinement sessions should be planned regularly (or as needed) by the SM (Scrum Master) : the objectives of the Backlog Refinement sessions are to ensure that backlog items to be considered for the Sprint Planning are in "ready for development state", i.e. that they:

1. Are clear enough to satisfy the Definition of Ready criteria
2. Small enough to be done within a sprint, or otherwise requiring to be broken down.

3.3 Sprint planning

The Sprints will have a duration of 2 weeks, and a Sprint Planning session will be held at the beginning of each sprint. In preparation for the Sprint planning (typically through a backlog refinement session), the PO should ensure that:

1. The Sprint goal is defined
2. The backlog items have been assigned a priority
3. The Stories candidate for the sprint are "Ready"

During the sprint planning session, the Team discusses and agrees on the scope for the upcoming sprint, comprising:

- Selection of the stories that will be implemented in the sprint.
- Estimation of the effort required for the stories in the sprint.
- Splitting of the stories into tasks.

As a general guidance, a story that cannot be completed within one sprint duration should be broken down into multiple stories.

One important output of Sprint Planning is that the upcoming sprint is linked with user stories and tasks that are agreed to be implemented.

Commented [WX7]: Highlighted the link here.

Commented [AT8R7]: Can you elaborate more this one? Lets have a short call and you can explain how you are proposing system (ADO) wise.

Commented [WX9R7]: Just add a clear description: when doing sprint planning, they need to consciously make the link.

3.3.1 Effort sizing

The sizing of effort for stories (story points) may follow any of the following methods:

- Fibonacci series (1, 2, 3, 5, 8, 13, 21)

3.4 Stand-ups

The stand-up session will be held each day of the sprint and facilitated by the Scrum Master. The participation of the PO is optional.
In this ceremony, each contributor in the Development Team will answer three simple project questions:

- What did you work on yesterday that contributes to meet the sprint goal?
- What are you working on today that will contribute to meet the sprint goal?
- Do you have any impediments/blockers or need any help?

As a guideline, a good stand-up session should be concluded in typically 15 minutes for teams of up to 10 people in size.

3.5 Retrospective

A Retrospective session shall be organized at the end of each sprint with the objectives of collecting feedbacks from the team in relation to issues and improvements needed as well as to satisfactory and well working aspects.

The Retrospective may utilize any appropriate tool (such as the one provided within ADO) and will generally adopt the following script:

- Set the stage.
- Gather data.
- Generate insights.
- Decide what to do.
- Close the retrospective.

4 DEVOPS - CONTINUOUS INTEGRATION & CONTINUOUS DELIVERY

The CI/CD process supports incremental and frequent commits to the shared codebase while ensuring quality via automated pipelines to build and test the code.

Based on the branch strategy described in section 5**Error! Reference source not found.**, the following procedures are foreseen for the code promotion into the main trunks.

Commented [CY10]: Section number to be adjusted

4.1 DEV Branch

The normal development process that ends up in a contribution to the Dev branch will comprise the following steps:

1. Developer creates a feature branch for the work item with User Story ID
2. Developer Implements changes and commits to feature branch
3. Developer creates Unit Tests and commits to feature branch
4. Developer creates PR of feature to DEV
5. The related pipeline writes the software version in the firmware configuration file before build, so that the version can be correctly reported by devices under test.
6. The related pipeline also launches the following checks:
    a. Unit tests
    b. Automated integration tests
    c. SAST
7. Dev Team Leader executes code review (or decides optional peer review assignment)
8. If all checks at steps 6. and 7. above are satisfied, Dev Team Leader approves / rejects PR
9. On PR approval, feature branch is merged into DEV

Notes:

- Dev Team leader is responsible to update ADO library variables with version number
- Dev build can be requested via ADO pipeline at any time for experimental tests
- Version number should include the build no. e.g. major.minor.x-build number and package name can be Productname_major.minor.x-buildnumber-Git hash the same but GIT hash will be different on each commit making build unique

Commented [KL11]: It should include a build number as well, so, full version will be major.minor.x-buildnumber While the package name can be Productname_major.minor.x-buildnumber-Git hash

4.2 QA & PROD branches

The QA branch is intended for the QA tests that allow to validate the release conformance for release.
The normal development process that ends up in a contribution to the QA branch will comprise the following steps:

1. Development Team Leader updates ADO Pipeline Library Variables for QA with current Release version number
2. Development Team Leader triggers PR from Dev to QA
3. The related pipeline launches the following checks:

   a. Unit tests
   b. Automated integration tests
   c. SAST
4. Development Team Leader executes code review (or decides optional peer review assignment)
5. BAT Tech Lead executes code review (or decides to delegate and trust Development Team Leader)
6. If all checks at steps 3., 4. and 5. above are satisfied, BAT DVT Manager approves PR
7. Dev is merged into QA branch
8. Dev Team Leader updates ADO Pipeline Library Variables for Dev/Release with new version number so that new builds will have new version number
9. BAT DVT team can download QA builds for DVT & SW Test
10. If the QA tests discover bugs, these are fixed on the Dev branch, and then merged back into QA. resubmitted for regression testing.
11. Once all QA tests are passed, the QA branch is merged into Main branch via a specific pipeline; Main gets tagged with the Release number.

Notes:

- QA build can be requested via ADO pipeline at any time
- Main build can be requested via ADO pipeline at any time
- Feature builds that may be needed from time to time for experiments, can be built via pipeline from a chosen feature branch.
- Secure code reviews and pen tests sessions by specialized security experts are recommended to be carried out at least twice per year.

5 SOURCE CODE MANAGEMENT

This section provides the main guidelines applicable to the activities related to the control and management of the source code, so as to ensure consistency and quality across the Development team.

5.1 Creation of a new repository

When creating a new Azure Repo, avoid using spaces in the repository name. This can prevent potential issues with URL encoding and command-line operations. For example, use AzureProjectRepo or Azure_Project_Repo instead of My Project Repo. Use _ to replace space

- **Create a New Empty Repository**
  - Go to your Azure DevOps project.
  - Navigate to **Repos** > **Files**.
  - Click on the drop-down next to the current repo name and select **New repository**.
  - Enter the new repository name (follow the example below).

	Prefix	Definition	Example
	BL	Bootloader	BL_mcu_boot - Bootloader Repo for each MCU
	LIB	Library	LIB_CLI_Auth - Software Library for CLI Authentication
	FW	Platform	FW_HP_Platform_PSoC63 - Firmware HP Platform based on PSoC63
	PRJ	Projects	PRJ_Etna_on_X2 - Project Repo stores Etna_on_X2 historical code base
	EXP	Explore/Experimental	EXP_Unit_Test - Explore & Experimental Unit Test Framework

  - Choose **Git** as the type and leave it as an empty repository.
  - Click **Create**.
- **Import the Template Repository**
  - After the new repository is created, click on **Import repository** (usually shown on the empty repo page).
  - In the **Clone URL** field, enter the template repository URL: https://dev.azure.com/ado-bat-uksouth-prod-01/FW_Platform/_git/FW_Repo_Template
  - Click **Import**.
  - Wait for the import process to complete. All template files and branches will be copied to your new repository.
- **Set the Default Branch**
  - Go to **Repos** > **Branches**.
  - Locate the develop branch.
  - Click the ... (More options) next to develop and select **Set as default branch**.
- **Update the README.md**

Commented [DD12]: - Avoid Spaces in Azure Repository Names:
When creating a new Azure Repo, avoid using spaces in the repository name.
This can prevent potential issues with URL encoding and command-line operations.
For example, use AzureProjectRepo or Azure_Project_Repo instead of My Project Repo. Use _ to replace space
- Include a README and .gitignore:
Always initialize your repository with a README file and a .gitignore file. The README provides an overview of the project, while the .gitignore file specifies which files and directories to ignore, keeping your repository clean.

- Open the README.md file in your new repository.
- Fill in all required sections and placeholders (e.g., {{PROJECT_NAME}}, {{TEAM_NAME}}, etc.) with your project-specific information.
- Commit your changes.

- **(Optional) Update Branch Policies and Pipelines**
  - If needed, configure branch protection policies for main, develop, and QA branches.
  - Set up CI/CD pipelines as required for your project.

5.2 Creation of a new pipeline

This section presents the recommended approach for creating Azure Pipelines for FW_Platform and Firmware Development projects. The objective is to facilitate a seamless handover between developers and DevOps, reduce potential misunderstandings, and expedite the pipeline setup process.

- **Local Build Validation**
  - Developers must ensure that the project can be built successfully in their local environment.
  - All required tools, dependencies, and environment variables should be clearly documented.
  - Developers should be able to demonstrate a successful local build as proof of readiness.
- **Update Project Documentation**
  - Developers update the project README.md to include the following key information.
    - **Required Tools:** List all compilers, toolchains, and utilities needed for the build, including Toolchain Versions, Installation Instructions and Expected Installation & compile paths.
    - **Build Commands:** Provide the exact CLI commands used for building the project locally.
    - **Generated Files:** Specify which output files (e.g., .hex, .bin, .elf) and paths are produced by the build process.
    - **Publish Artifacts:** Indicate which files should be included as pipeline publish
- **Create a User Story**
  - Developers create a User Story (US in Azure DevOps to request pipeline creation.
  - The User Story should reference the updated documentation and any special requirements.
  - The US/Task remains under the developer's name until a demo is provided to DevOps and both sides are aligned. The demo can be done offline and does not require a group meeting.

- **Demo and Handover to DevOps**
  - Developers provide a demo (can be a short video, screenshots, or a live session) to DevOps, showing the local build process and results.
  - This step ensures DevOps can fully understand the build process and requirements before starting pipeline setup.
  - Once both sides are aligned, the US/Task can be officially assigned to DevOps.

- DevOps will read and review the Tech Documentation and verify that the local build steps are reproducible without any issues.
- If any information is missing or unclear, DevOps will communicate with developers for clarification.

- **Pipeline Setup**
  - DevOps creates the Azure Pipeline based on the provided documentation and validated local build steps.
  - The pipeline typically includes:
    - **Source Code Checkout:** Pulling the latest code from Azure Repos.
    - **Environment Setup:** Installing required tools and setting up the build environment (e.g., using Docker if needed).
    - **Build & Compiling Execution:** Running the documented build commands or procedures / build scripts
    - **Artifact Publishing:** Collecting and publishing the specified output files to Azure Pipeline or Azure Artifacts
    - **Optional Steps:** Integrating code quality checks (e.g., SonarQube), running unit tests, or signing firmware or Firmware Merge if required.
- **Pipeline Build Validation**
  - DevOps runs the pipeline and verifies that it produces the expected results.
  - Any issues encountered are communicated back to the development team for resolution.
  - If problems occur, developers and DevOps should collaborate (e.g., online debugging)
  - Both DevOps and developers should timely add comments to update the progress in the US for transparency.
- **Handover and Maintenance**
  - Once validated, the pipeline is handed over for regular use.
  - Future changes to the build process or requirements should be reflected in the documentation and communicated to DevOps for pipeline updates.

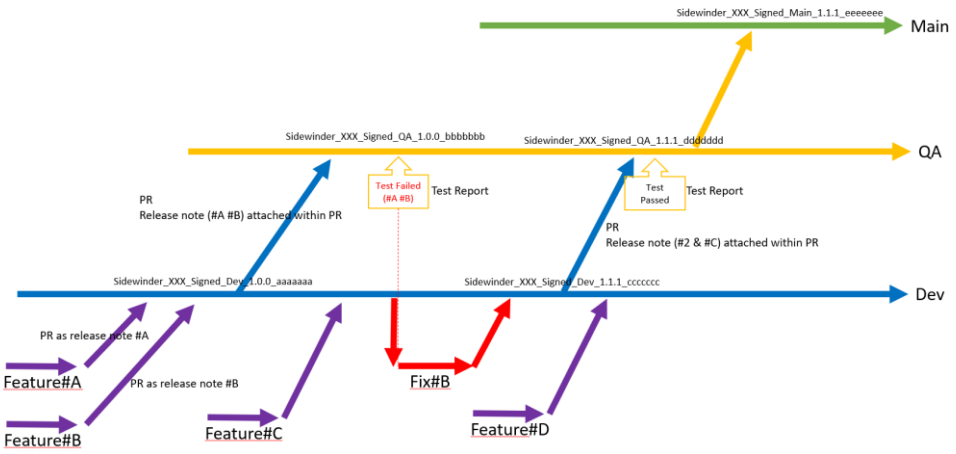
5.3 Branch strategy

The codebase management approach chosen involves the use of three main branches:

- **Development (Dev)**
  - The integration branch for features and bug fixes, used for staging and testing. The Development branch contains all the features developed until the most recent sprint. The features are divided into features of the past sprints (that are already tested and validated into the QA branch) and features of the most recent sprint (that have to be validated into the QA branch). All the tests, that includes Unit test, manual and automatic functional tests, are executed inside this branch. Specific feature branches and bug fix branches created during the sprint to implement the user stories are derived from the development branch.
- **Staging (QA)**
  - The QA branch contains a CI/CD tested and integrated firmware which needs to be approved by BAT team. The branch is used for quality assurance testing before merging into the main branch.
- **Main (Prod)**

- The stable branch containing production-ready code. The QA branch contains a CI/CD tested and integrated firmware which needs to be approved by BAT team.

Example of Branch Management



5.4 Hot fix Management

Step 1

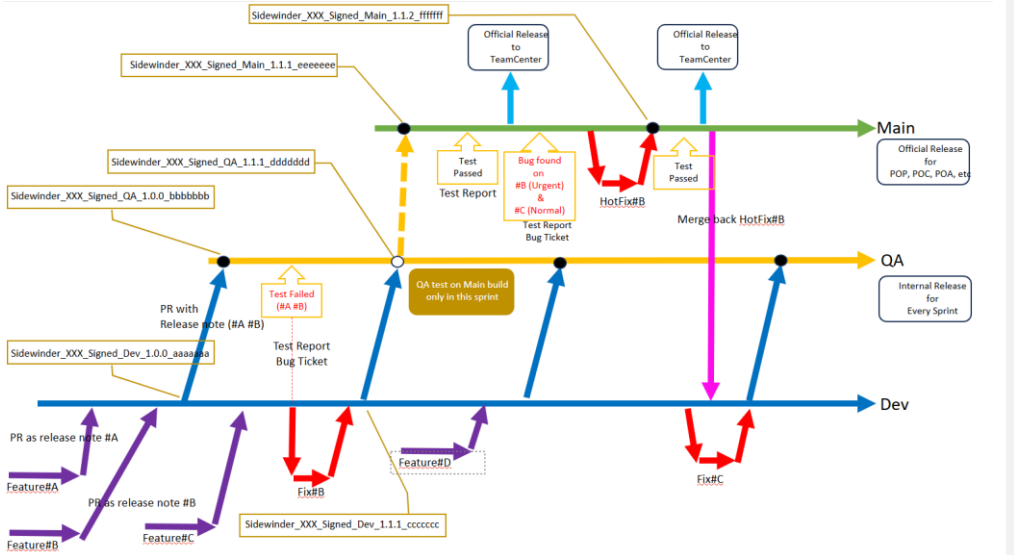
- A bug is found within the Main branch.

Step 2

- A hot fix branch is created starting from the main branch, once the bug is fixed and tested the hot fix branch is merged in to the Main branch.

Step 3

- A branch aligned with the latest release is created from the Main branch and merged into the QA and Dev Branches.



5.5 Feature Branch Naming

Branch naming shall follow an agreed naming convention, e.g. of the type:

<developer name>_[feature/bug/hotfix]_<work item ID>_<title>

5.6 Branches naming conventions

Regarding the naming conventions of branches and firmware, the following convention was decided upon:

Type of branch	Branch name
Feature branch	**feat_<US_ID>**
Bug fix branch	**fix_<BUG_ID>**

Commented [AT13]: Check with the Platform leads whether or not the naming convention is practical to adopt or the process needs to adapt

5.7 Branch Policies

For the creation of branch policies in ADO, reference can be made here:
Set and manage branch policies - Azure Repos | Microsoft Learn

Branch policies shall be configured for three main branches, Dev, QA and Prod.

Reviewers should be managed through ADO groups rather than individual user accounts to simplify maintenance and ensure role-based governance.

The policy should require approvals from the corresponding stakeholder groups so that changes are reviewed and approved by representatives from different roles before being merged.

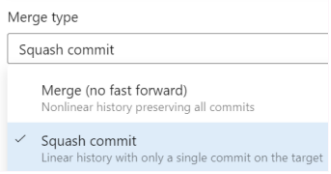
Commented [WX14]: Changed the plain URL link to one with name and added a brief description about how branch policies are applied, because the roles responsible for approval in E2E project are defined in PDLC.

5.8 Commit Best Practices

Guidelines on commit messages shall be followed in order to have a common and uniform standard that facilitates human and machine interpretation.

When Merge feature branch to dev, the commit message is controlled by Pull Request Template, so, it's not applicable to this section.

Regarding feature branch commit, there are two cases, which depends on Pull Request merge type, Squash or Merge.



- Merge Type
  Developer wants to merge feature branch commit history to dev branch. The commit message in feature branch should follow rule in this section.

- Squash Type
  Developer Does **NOT** want to merge feature branch commit history to dev branch, which means only Pull Request commit message to dev branch. Developer does not need to follow this rule exactly.

Most of time, Squash Type should be used.

Commented [KL15]: I added these

The main guidelines are:

- Separate subject from body with a blank line
- Limit the subject line to 50 characters
- Capitalize the subject line
- Do not end the subject line with a period
- Use the imperative mood in the subject line
- Wrap the body at 72 characters
- Use the body to explain what and why vs. How

Useful references can be found are:

- The **Conventional Commits specification** is a lightweight convention on top of commit messages. It provides an easy set of rules for creating an explicit commit history; which makes it easier to write automated tools on top of.
- **commitlint** is a tool (that can be integrated into the DevOps pipeline) to configure and to enforce commit conventions.

5.9 Semantic Versioning Structure

We follow the Semantic Versioning 2.0.0 Structure to clearly communicate software/firmware versioning and manage SCM tags, artifact feeds, dependencies, releases, and QA process.

Commented [AT16]: This structure is specific for the application. For BL there will be another section providing the coverage for the versioning structure.

Typical standards for the semantic versioning structure follow the general format of:

MAJOR.MINOR.PATCH[-Pre-release Tag][+Build Metadata] with the following conventions:

- **Major Version:** Significant or Incompatible Changes: The changes that are not backward compatible such as changing the communication protocol or hardware interface.

- **Minor Version:** Mainly for new Features: Incremented for backward-compatible new features, such as adding a new temperature control mode.
- **Patch Version:** Bug Fixes, Incremented for backward-compatible bug fixes, such as correcting a temperature display error.
- **Additional Metadata** To capture the following metadata, we propose an ad-hoc versioning method using pre-release tags and build metadata.

Pre-release Tags: Use a hyphen (-) to denote pre-release versions.
Purpose: Pre-release tags indicate that the version is unstable and may not meet the intended compatibility requirements.

They are typically used for testing or preview versions.

Impact: Pre-release versions have lower precedence than the associated normal versions with the same major, minor, and patch numbers. For example, 1.0.0-alpha has lower precedence than 1.0.0.

-alpha Early testing phase.
-beta Feature complete but may contain known issues.
-rc Release candidate, potentially final version unless significant bugs emerge.
-hotfix Urgent bug fixes.

Build Metadata: Use a plus sign (+) to denote build metadata.
Format: Use a plus sign (+).
Examples: 1.0.0+buildid, 1.0.0-alpha+exp.sha.5114f85.
Purpose: Build metadata provides additional information such as build ID, timestamp, etc. This information helps distinguish between different build versions but does not affect the version precedence.
Impact: Build metadata does not affect the version precedence. For example, 1.0.0+buildid and 1.0.0 have the same precedence.
+buildid
+maintenanceid
Unique identifier for the build/main

6 PULL REQUESTS

Pull Requests (PR) are a verification gate that shall be carried out upon any change to be contributed into the main codebase branches.

The Pull Requests trigger two main forms of code qualification:

1. Code Reviews
2. Automated checks (i.e. linting, compilation, Unit tests, Integration tests, SAST, etc)

Before creating a PR, Developers shall:

1. Verify that the code conforms with the coding standards and passes the linter check
2. Verify that the code compiles and runs without errors
3. Write and/or update tests to cover the changes and make sure all new and existing tests pass
4. Write and/or update the documentation to match the changes
5. Write and/or update the code comments to explain the changes and make sure all comments in English

Commented [AT(17]: Technical consideration required on this section

Commented [AT18]: Suggest that we add SonarQube here: Pull requests shall be subject to SonarQube automated scanning where the quality gate is enabled. At the POC stage, the quality gate will block any pull request containing High or Blocker-level issues. At the POA stage, the quality gate will block pull requests containing Medium, High, or Blocker-level issues. Developers must resolve the blocking issues before the pull request can be completed. A bypass of the quality gate shall only be allowed in emergency cases. The Project PM may request DevOps to approve a temporary bypass, with proper justification and traceability recorded, before the PR can be completed.

Commented [WX19]: Added one more item to check, and highlighted the requirement in English.

6.1 Guidelines on PRs

In creating a PR, the Developer should apply the following general guidelines:

1. Focus:
   - The PR should be related to a single reason for code change, typically a user story. They should not mix parts of code not related to an identifiable story or task
2. Small size:
   - Related to the point above, PRs, should avoid to be too large in size, as the consequent code review process and deployment may be long and break the principle of agile and incremental progress.
3. Description:
   - PR descriptions should follow a standardized format & template for better readability. Developers should use the PR Template in section 6.2 which is already configured in the ADO.

6.2 PR Template

A PR template shall be created in order to standardize the PR contents and format.

A .md template file can be created (named *pull_request_template.md*) and placed in one of following three locations, so that ADO can detect it and auto-format new PRs accordingly:

- The root folder of the repository
- A folder named *.azuredevops* that is contained in the root folder of the repository
- A folder named *docs* that is contained in the root folder of the repository

The following is a template checklist for the contents of a PR, that is recommended for standardization. It is taken, with adaptations, from [**Error! Reference source not found.**]:

I. Description

A summary description of the changes (bug or feature), its motivation, and a summary of the solution. When writing the description of your changes, add references to any features, tasks or bugs you worked on. Imagine the reader is new to the project and doesn't have any background.

II. Steps to Reproduce Bug and Validate Solution

Commented [AT20]: To be checked with QA. Confrim the new version and validate against the template in the SDLC

This section should provide details on how to reproduce the bug, in case the PR addresses a bug (not applicable if related to a feature or story).

III. PR Checklist

Section to confirm that the essential checklist is satisfied before triggering the PR:

1. Verify that the code conforms with the coding standards and passes the linter check
2. Verify that the code compiles and runs without errors
3. Write and/or update unit tests to cover the changes and make sure all new and existing tests pass
4. Write and/or update the documentation to match the changes

IV. Breaking change

Indication of whether the PR introduces a breaking change or not, and description of the impacts to be considered for the depending code. Give some details around anything that might break with this change. You can put None Expected if you don't expect it to break anything

V. Testing

Indications and instructions relevant to the testing and validation of the code. For example How did you test it? Where are the test results? Have there been any test reports? Is it a QA or Production release?

VI. Dependencies

Any known dependencies that are important to this PR. Tell us about it.

6.3 Code reviews

As a general guideline, code reviews should be done on every PR, or check-in to a shared branch.

Code reviews are useful not only to identify potential defects in the code, but also to promote a shared understanding of the project's code across the development team.

The reviewer's objectives should be to check the following attributes of the code under review:

- the adherence to the reference coding standards
- the correctness of the business logic
- the correctness of any new or changed tests.
- the "readability" and maintainability

As a guideline, the code review can use the following checklist:

☑ Does the code work? Check whether function and logic are correct.

☑ Are functions, methods, and variables adequately named?

☑ Are names semantic and meaningful?

☑ Is it well tested?

☑ Are there unit tests, and they have good quality? Do they test the functionality?

☑ Do tests reach acceptable coverage for the project?

☑ Is the code clear and easy to understand?

☑ Does it meet the coding standards and style guides?

☑ Is there any duplicated code?

☑ Is there any commented or unnecessary code?

☑ Does the code take the most out of frameworks and language? Is there any custom implementation of native or already-existing functions?

☑ Are there "code smells", i.e. non-optimal design or implementation that can turn into technical debt?

☑ Is documentation on functions, methods, classes, contexts, and behaviors adequate?

☑ Is the code as modular as possible?

☑ Are the critical spots adequately logged?

☑ Does the code consider failures? Is it just considering the happy path?

☑ Are there better or simpler solutions?

☑ Is there any performance issue?

☑ Are input data sanitized?

☑ Is sensitive information being encoded or encrypted?

Commented [AT(21]: Technical review/validation required

Commented [KL22R21]: I think this is fine. This is just a guide line. TBH, during practice, we cannot check every item, normally, we focus on coding rule, logic, naming, understandable, mandatory comments,

7 CODING STANDARDS

The referenced coding standards document (BAT FW Coding Standard) outlines the coding standards that should be followed throughout the Software Development Life Cycle (SDLC) to promote the consistency, readability and collaboration across development teams.

8 DEVELOPMENT ENVIRONMENT STANDARDS

A standardized development environment shall be established and maintained across development teams to reduce onboarding effort, minimize environment setup time, ensure consistent build outputs, and support reliable and efficient CI/CD execution.

The referenced Development Environment Standards document Platform_Developer_Env_Setup_20250916 defines the required tools, configurations, setup procedures, and maintenance expectations to be followed throughout the Software Development Life Cycle (SDLC).

The best choice for Pipeline build is Linux command line mode, but most developers are using Windows to develop, so, a standard env for multi OSs are defined with cmake to build target and edit with VSCode. And In VSCode, it can integrate building , debugging, AI and Sonar Qube.

Tool Name	Version in Ubuntu 24.04	Version in Win11 X64	MacOS TBD	Comments
VSCode	Latest	Latest		For developer (Pipeline not need)
Git for windows	-	>= V2.44		Windows only
Jlink	V8.24	V8.24		Some MCU may need latest or old version.
Python	V3.12	V3.12		For mcuboot signing
Cmake	V3.28.3	V3.28.3		Compile
Make	V4.3	V4.2		Compile
ARM GCC	V14.2Rel1	V14.2Rel1		Compile
X86 GCC	V13.3.0	V13.2.0-posix-seh-ucrt		Unit test (No 13.3 in windows)
Gcovr	V7.0	V8.0 (Python pip install)		Unit test
Ruby	V3.2.3	V3.2.9		Unit Test
Ceeding	V1.0.1	V1.0.1		Unit Test
Azure tool (azcli and azure-devops-cli-extension)	?	>= v2.67.0 >=extension > 1.0.1		For downloading lib package

These standards are intended to improve consistency, maintainability, reproducibility, and collaboration across development teams.

Commented [KL23]: @Kevin Yu @Ahsan Tahir (External) I added this chapter for standard develop environment. the description is similar as coding standard so far. Please check if it's ok.

Commented [KL24R23]: @Andrew Tung FYI

Commented [CY25R23]: @Kent Lee Could we have a brief description of how this standard environment would be e.g. what IDE, what compiler @Ahsan Tahir (External) I think we need to add the link of the instruction document. And because of that, we need to have a proper version control and storage of such instruction document as well.

Commented [KL26R23]: @Kevin Yu agree, pls see if it's ok now?

9 CODE-RELATED SECURITY PRACTICES

The recommended security practices related to the code structure are provided in the guidelines document [**Error! Reference source not found.** BAT_IOT_SSDLC_Nov22_AL]

Commented [DD27]: It would be better to add a chapter on 'Coding Standards / Convention and Best Practices' before Chapter 8 to ensure code consistency and maintainability.

7 Coding Standards, Convention and Best Practices

General Coding Standards
- Consistent Indentation and Formatting:
Use consistent indentation to improve code readability.

- Header File Inclusion:
Ensure correct case when including header files, as Linux environments are case-sensitive. Incorrect casing can lead to build failures.

#include "MyHeader.h" // Ensure correct case

! Real case occurred on Boreas project, where the inclusion of header files with incorrect casing caused the Azure DevOps pipeline build fail.
Since the ADO pipeline runs in an Ubuntu Linux environment so it is very case-sensitive. If the case of the included .h file does not match exactly, the file cannot be found, leading to build failures.

e.g. #include app_taskInit.h --> app_taskInit.h

- Naming Conventions:
Use meaningful and descriptive names for variables, functions, and constants. Use camelCase or snake_case for variables and functions, and UPPER_CASE for constants and macros.

- Comments and Documentation:

etc....

Commented [AT28R27]: Coding Standard section has been added.

Commented [AT29]: To be reviewed thoroughly and understand how we can implement the security practices into FW development.

10 PROCESS RELATED SECURITY PRACTICES

10.1 Secrets management

It is important to observe best practices for ensuring that secrets and sensitive data such as usernames, passwords, API keys, etc are not disseminated improperly in the codebase.

10.1.1 General guidelines

Secrets shall be kept in separate local configuration files that are not checked in to the repo by adding the files to the .gitignore list. Such files may be identified with extensions such as, for example, .private.config.

Note that even if a secret were only pushed to a feature branch and never merged into master, it will still be part of the git history. To remove any sensitive data and/or regenerate any keys and other sensitive information added to a repo, reference can be made to the git filter repo approach.

Separate secrets configurations should be maintained for each environment (dev, staging, prod).

10.1.2 Managing secrets in ADO on a per-branch basis

In ADO's pipelines for CI/CD pipeline variables can be used for secrets management, but they have the following disadvantages:

1. Pipeline variables are managed outside the code that references them. This makes it easy to introduce drift between the source code and the secrets, e.g. adding a reference to a new secret in code but forgetting to add it to the pipeline variables (leads to confusing build breaks), or deleting a reference to a secret in code and forgetting to remote it from the pipeline variables (leads to confusing pipeline variables).

2. Pipeline variables are global shared state. This can lead to confusing situations and hard to debug problems when developers make concurrent changes to the pipeline variables which may override each other. Having a single global set of pipeline variables also makes it impossible for secrets to vary per environment (e.g. when using a branch-based deployment model where 'master' deploys using the production secrets, 'development' deploys using the staging secrets, and so on).

A solution to these limitations is to manage secrets in the Git repository jointly with the project's source code, but not in plain text.
The approach is to add an encrypted version of the secrets to the repository and enable the CI/CD agents and developers to decrypt the secrets for local usage with some pre-shared key.
In this way we ensure both a secure storage for secrets and side-by-side management of secrets and code.

```
# first, make sure that we never commit our plain text secrets and generate a strong encryption key
echo ".env" >> .gitignore
ENCRYPTION_KEY="$(LC_ALL=C < /dev/urandom tr -dc '_A-Z-a-z-0-9' | head -c128)"

# now let's add some secret to our .env file
echo "MY_SECRET=..." >> .env

# also update our secrets documentation file
cat >> .env.template <<< "
# enter description of your secret here
MY_SECRET=
"

# next, encrypt the plain text secrets; the resulting .env.enc file can safely be committed to the repository
echo "${ENCRYPTION_KEY}" | openssl enc -aes-256-cbc -md sha512 -pass stdin -in .env -out .env.enc
git add .env.enc .env.template
git commit -m "Update secrets"
```

When running the CI/CD, the build server can now access the secrets by decrypting them.
For ADO, configure ENCRYPTION_KEY as a secret pipeline variable and then add the following step to azure-pipelines.yml:

```
steps:
  - script: echo "$(ENCRYPTION_KEY)" | openssl enc -aes-256-cbc -md sha512 -pass stdin -in .env.enc -out .env -d
    displayName: Decrypt secrets
```

To allow pipelines to manage all secrets in one location, it is also recommended to use variable groups linked directly to Azure key vault.

10.1.3 Credentials scanning

Credential scanning is the practice of automatically inspecting a project to ensure that no secrets are included in the project's source code.

From a software design perspective, the code should be independent of the runtime configuration that will be used to run it, and that runtime configuration includes secrets. As such, there should be a clear boundary between code and secrets: secrets should be managed outside of the source code and credential scanning should be employed to ensure that this boundary is never violated.

Credential scanning shall be enforced as part of the CI process to ensure that no secrets ever get merged to a project's main branch.
To this purpose, an ADO pipeline shall be configured to enforce build validation for PRs before merging to the main branch.

Among the possible credential scanning tools to be used, examples are:

- Yelp detect secrets
- CodeQL
- Git-secrets

11 SDLC RACI

The following table summarizes the RACI matrix for the main SDLC activities and the respective roles and responsibilities.

(R = Responsible, A = Accountable, C = Consulted, I = Informed)

Roles

ACTIVITIES	BAT Product Manager	BAT Product Technical Lead	BAT Platform Owner	BAT Platform Development Team	BAT Platform DevOps	Platform Architect	ODM Product Manger	ODM Technical Lead	BAT Test Manager
Product Requirements Definitions	A	R	C	C	C	I	R	I	C
Platform Backlog Creation (Epics, Features)	C I	C I	A R	C	C	C	I	I	C
Platform Design	I	I	I	R	I	A R	C	C	C
Product Design	I	C	I	C	C	I	A	R	C
Platform Development	I	I	A	R	C	C I	C	C	C
Product Development	C	C	C I	C I	I	I	A	R	C
Platform QA Testing	I	C I	C	C	C	I	I	I	A R
Product QA Testing	I	C	C I	C	C	I	C	C	A R
Platform Release Management	I	I	A	R	R	I	I	I	I
Platform Maintenance (Bug fix, Support EoL)	I	C	A	R	C	R	I	I	R
Product Release Management and Deployment	A	R	C	C	C	C	R	R	I

12 RELEASE CADENCE

The frequency of a release has been set at **every Sprint**.

During the sprint, every feature (User Story) to be developed is associated at a specific feature branch, which is derived from the **dev branch.**

Every time a User Story is completed, a set of tests related to that specific feature is executed by the test team.Test are related on ADO to a specific user story, to allow traceability.

When a functionality is tested, then it is merged through a PR to the **dev** branch.

One or Two days before the end of the sprint, when all features are completed and tested, then a PR to the **QA** branch is made, in order to execute a full **CI/CD pipeline**.

The pipeline contains the following stages:

- **Compile the source code using IAR toolchain**
- **Build the firmware binary file**
- **Execute the Unit Tests and the code coverage**
- **Execute SAST and DAST analysis (note: NOT IMPLEMENTED YET, UNDER ANALYSIS)**
- **Generate Doxygen documentation**
- **Merge the firmware binary with bootloader and soft-device (only for Nordic MCUs)**
- **Create OTA package to allow wireless firmware update**
- **Execute automatic tests on the device using a specific test rig (Raspberry Pi)**
- **Prepare all the reports**

At the end of the pipeline, if all tests are passed, then are produced several artefacts:

- **Complete firmware binary**
- **OTA package**
- **Code documentation**
- **Test reports and documentation (Release note will be uploaded on Share point)**

In a **QA** pull request, all the US related to the functionalities implemented during the sprint are linked as Work Items and, if the PR is completed successfully, **the US are automatically marked as closed**. This because, after a full CI/CD pipeline, the Definition of Done has to be considered fully respected.

In the event that a bug occurs during the development phase of a sprint-related feature, a bug item will be created on ADO and added as a Task for the **current sprint**.
By the end of the sprint, if the bug isn't already solved, then is carried over to the next sprint as a **fix** of this the new sprint and resolved in the same.

The bugs are created on ADO and linked to the related US and Tests to maintain traceability
Bugs, as shown in the **Release Note Template**, are classified as:

- **Urgent-Blocking Bug**
- **Major Bug**
- **Minor Bug**

If **Urgent-Blocking** Bugs or a **Major** Bugs aren't solved by the end of the sprint, the PR to QA is not made and the firmware release remains on the **DEV** branch.
In that case, the bugs are pushed in the Next Sprint Plan and the release to **QA** happens at the end of that sprint, and will include the functionalities of both the sprints.

So, the specific bug is handled as:

- Bug reporting and tracking on ADO (bug related to a US)
- Bug resolution
    - *Close the bug -> mark the bug as Resolved until the next PR closes it*
- Non-resolution of the bug
    - *Reassignment of the bug to the next sprint*

At the end of the pipeline, if all tests and stages are completed successfully, a PR on **QA** branch is made and the firmware is released to allow BAT-side team to do their internal quality checks.
Features are developed within the "Feature" branch, and if the internal tests run and are successful, the new features are placed within the **DEV** branch.

Within the **DEV** branch, automatic tests are run and if issues such as pipeline failure are detected, subsequent PR on the QA branch does **not occur**.
Otherwise, the branch is pushed to the **QA** branch for acceptance testing by the BAT team.

At the end of the process, if the QA branch is officially approved, then the firmware is released into the **Production (Release)** branch, which is the "stable" mainline of the product and can be considered ready for production.

Notes: The Release Notes are insert inside appropriate directory in Share point (the link will be shared in private chat). Below is present an example of directory tree:

SharePoint BAT
|_ Documents
|_ Inside Channel
|_ Name of project
|_Release Notes
|_ Sprint "XX"

13 STORAGE OF SECRETS AND SIGNING

13.1 Safe secrets storage

Passwords should never be stored in unencrypted forms for confidentiality reasons. Moreover, protected passwords could be easily stolen by an attacker leveraging common vulnerabilities such as SQL injection or from artefacts such as log files, dumps, and backups.

13.2 General guidelines

Secrets shall be kept in separate local configuration files that are not checked in to the repo by adding the files to the .gitignore list. Such files may be identified with extensions such as, for example, private.config.

Note that even if a secret were only pushed to a feature branch and never merged into master, it will still be part of the git history. To remove any sensitive data and/or regenerate any keys and other sensitive information added to a repo, reference can be made to the git filter repo approach.

Separate secrets configurations should be maintained for each environment (dev, staging, prod).

13.3 Managing secrets in ADO on a per-branch basis

In ADO's pipelines for CI/CD pipeline variables can be used for secrets management, but they have the following disadvantages:

- Pipeline variables are managed outside the code that references them. This makes it easy to introduce drift between the source code and the secrets, e.g. adding a reference to a new secret in code but forgetting to add it to the pipeline variables (leads to confusing build breaks), or deleting a reference to a secret in code and forgetting to remote it from the pipeline variables (leads to confusing pipeline variables).

- Pipeline variables are global shared state. This can lead to confusing situations and hard to debug problems when developers make concurrent changes to the pipeline variables which may override each other. Having a single global set of pipeline variables also makes it impossible for secrets to vary per environment (e.g. when using a branch-based deployment model where 'master' deploys using the production secrets, 'development' deploys using the staging secrets, and so on).

A solution to these limitations is to manage secrets in the Git repository jointly with the project's source code, but not in plain text. The approach is to add an encrypted version of the secrets to the repository and enable the CI/CD agents and developers to decrypt the secrets for local usage with some pre-shared key. In this way we ensure both a secure storage for secrets and side-by-side management of secrets and code.

To allow pipelines to manage all secrets in one location, it is also recommended to use variable groups linked directly to Azure key vault using the "Secure files".

```
# first, make sure that we never commit our plain text secrets and generate a strong encryption key
echo ".env" >> .gitignore
ENCRYPTION_KEY="$(LC_ALL=C < /dev/urandom tr -dc '_A-Z-a-z-0-9' | head -c128)"

# now let's add some secret to our .env file
echo "MY_SECRET=..." >> .env
```

```
# also update our secrets documentation file
cat >> .env.template <<< "
# enter description of your secret here
MY_SECRET=
"

# next, encrypt the plain text secrets; the resulting .env.enc file can safely be committed to the repository
echo "${ENCRYPTION_KEY}" | openssl enc -aes-256-cbc -md sha512 -pass stdin -in .env -out .env.enc
git add .env.enc .env.template
git commit -m "Update secrets"


When running the CI/CD, the build server can now access the secrets by decrypting them.
For ADO, configure ENCRYPTION_KEY as a secret pipeline variable and then add the following step to azure-pipelines.yml:

steps:
  - script: echo "$(ENCRYPTION_KEY)" | openssl enc -aes-256-cbc -md sha512 -pass stdin -in .env.enc -out .env -d
    displayName: Decrypt secrets
```

13.4 Secure files

Secure files give you a way to store files that you can share across pipelines. Use the secure files library to store files such as:

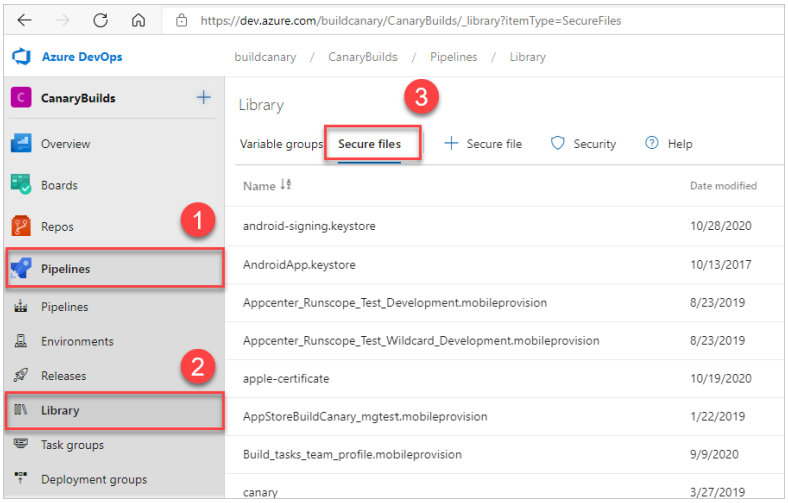
- signing certificates
- SSH keys

These files can be stored on the server without having to commit them to your repository.
The contents of the secure files are encrypted and can only be used when you consume them from a task. Secure files are a protected resource. You can add approvals and checks to them and set pipeline permissions. Secure files also can use the Library security model.

The size limit for each secure file is 10 MB.

13.4.1 Add a secure file

1. Go to Pipelines > Library > Secure files.



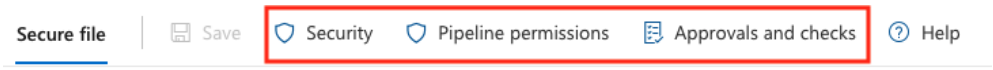
2.Select + **Secure file** to upload a new secure file. Browse to upload or drag and drop your file. You can delete this file, but you can't replace it.

3. Add permissions to your file:

Apply security role restrictions for all files from the **Security** tab at **Pipelines** > **Library**.
To add permissions for an individual file, in the file's edit view, select **Pipeline permissions** to set per-pipeline permissions.

Or, select **Security** to set security roles.

You can also set Approvals and Checks for the file. For more information, see Approvals and checks.



13.4.2 Signing and publishing

Once an application has been successfully developed, the next step may be publishing it and sharing it with others. In this case, software applications must be signed with a cryptographic signature before being distributed to others.
It is important to observe best practices for ensuring that secrets and sensitive data such as usernames, passwords, API keys, etc. are not disseminated improperly in the codebase.

Code signing makes feeling safe a user who chooses to use a certain software application, provided that the application has been developed by reputable developers.
Code signing ensures that nothing has changed since the developer signed the code, which means there is no chance for an attacker to change the code that was written by the developer.

In summary, code signing makes applications safe for users. However, there are also some limitations, as code signing does not protect users from attacks based on dynamic content, which can occur, for example, when the application loads data from various third-party sources.

14 RELEASE NOTES

14.1 BASIC RELEASE NOTES TEMPLATE

Release Notes – Sprint No

Version	Release Number	Date	Approved By
			BAT

14.2 SOFTWARE RELEASE INFORMATION

- Link to the sprint on ADO -
- Repository – Sidewinder
- Branch name - dev
- Pull request -
- Unit test reports –
- ADO test plan –
- Automation test report -
- Manual test reports –

14.3 User stories

- User story link

14.4 Overview

14.5 Features

➢ Feature Name
- **Description**

14.6 Bug Fix and Code review

➢ Bug Name
- **Root cause**
- **Solution**
- **Bug video (If Any)**

14.7 Generic Information

The documents related to the additional features, design and DevOps guide are updated on the MC_Mimosa wiki page.
Link - Mimosa Wiki - Overview (azure.com)

Commented [AT30]: How should we link the Test reports back to the release notes. Are we producing at the correct development stage? Identify the correct process of incorporating the Test reports in the release notes Check with DevOps

Commented [AT31R30]: Automation test report, N/A for platform however, it applied for the product.

Commented [AT32]: Adjust the heading numbering

Commented [AT33]: Confirm with Devs whether we need it or not and modify the description

15 SOFTWARE TEST STRATEGY

15.1 Software test input and test scope

15.1.1 Software test input

Software documents form the cornerstone of both software development and testing. These documents are captured in two primary categories: Software Specification Documents and Referenced Documents. Their accuracy, completeness, and traceability directly influence the quality of the final product.

- Software Specification Documents:
  - User Story: The master requirement document for software development purpose to cover the aspects of the electronics, the relevant details of the overall system definition.
  - User Flow: UIUX file which Outline the sequence of interactions between users and the system/devices.
  - BLE Profile: Define Bluetooth Low Energy communication protocols for device interoperability.
  - CLI Command List: Define the CLI command for internal testing and factory testing, not visible for end user.
  - Error Code List: Catalog predefined error conditions and their handling mechanisms.

These documents are collaboratively authored and maintained by cross-functional teams, including the software team, device design team, and Application design team, etc., Regular updates before locked specification are critical to reflect evolving requirements or stakeholder feedback.

- Referenced Documents:
  - Product Requirement Specification(PRS): Overview for product high-level feature list from each dev function.
  - EE-Related Documents: States electrical and embedded system configurations, e.g., power management thresholds.
  - Software testing learn lesson list
  - Other related document

These documents help developers and testers understand broader system dependencies and design intent.

15.1.2 Software test scope

Software test scope is bounded by the main software specification documents, ensuring that every documented requirement is validated.

Mandatory Coverage for Main Software Specification Documents
Software testing engineers need to study these main software specification documents include User Story, User flow, Error codes list and BLE profiles to derive test purposes from these documents, design at least one test case to promise suitable test coverage for each single requirement.

Role of Referenced Requirements
Referenced documents inform risk-based testing but are excluded from mandatory coverage.
For example, the PRS may highlight market-specific performance thresholds, guiding load-testing priorities test, and EE related thresholds, testing these aspects is discretionary, as they often involve cross-team or system-level validations beyond the software test team's ownership.

Traceability and Accountability
A Requirements Traceability Matrix is essential to prove test coverage.
Test plan must ensure traceability between requirements and test cases to guarantee no gaps in verification, each test case in the test plan must reference a main requirement ID, ensuring accountability.
Deviations (e.g., untestable requirements due to ambiguity) are expected be flagged for requirement refinement.

15.2 Software test environment, tools and methodology

15.2.1 Software test environment

Software testing is generally conducted at room temperature.
Typically, error codes triggered by temperature points (e.g.: -20℃, 0℃, 45℃) are executed using the Command Line Interface(CLI).
But in some special test cases, software test will use climate chamber to simulate external temperature for error code test within some sprint test.

15.2.2 Software test consumable

Software testing is generally conducted without using real Vapor Pod and HP cigarettes.

- Typically in Vapor project, conduct test by using dummy Pod with related resistance (e.g.: 0.8Ω ~ 1.2 Ω).
- Typically in HP project, device are being heated with nothing, so-called dry burnt.

But in some special test cases, software test will use real Vapor Pod and HP cigarettes based on actual needs.

15.2.3 Software test tools requirement

To aid in the testing process and guarantee correctness, a variety of hardware instruments and software tools are used in software testing. The utilization of these technologies increases test result reliability in addition to testing efficiency.

- Management system
  - Azure DevOps
  - Teamcenter
- Hardware instruments
  - Test Laptop
  - Test Smart Phones (iPhone or Android phone with different brands and different OS version depending on detail project needs)
  - USB Charger (PD charger/ non-PD charger …)
  - Wireless Charger
  - USB Power Meter
  - Multimeter
  - CLI Cable
  - Climate Chamber
  - Raspberry Pi
  - Smoking machine (Vapor project)
- Software tools
  - Serial Terminal Emulator (e.g.: Putty, teraterm…)
  - PC tool
  - nRF Connect
  - BAT CLI Console
  - BAT Engineering App (BAT internal use only)

15.2.4 Software test sample quantity requirement

The following table only provides a recommended test sample quantity.
The test sample quantity already include the Tier 1 supplier software testing need.
The actual sample quantity depends on the specific project situation, such as the project build, device configuration and the functions which need to be verified.

Software test sample quantity requirement					
	POP build	POC build	POA build	DP build	SP build
Function test	1	10	10	10	10
Automated test	-	-	10	10	10
Stability test	-	-	20	20	20

15.3 Software test type

Test Type	Test Activity	Ownership	Frequency
Code quality test	Code quality testing involves executing code to ensure it behaves as expected, BAT uses SonarQube as standard tool for code quality test.	Dev	Each pull request
Unit test	Unit testing is the process of checking small pieces of code to ensure that the individual parts of a program work properly on their own.	Dev	Each pull request
Smoking test	Smoking test is a preliminary software testing method that evaluates whether an application's most fundamental functions work correctly. It will cover some test cases which couldn't be done by automated integration test.	Dev	Each QA branch release Each Main branch release
Regression test	Regression test will be performed which is the re-execution of a fixed subset of test cases. Regression test will include bug fix verification test suit, sanity test suit, new feature test suit.	QA	Each QA branch release
Automated test	Automated testing will comprise the setup of a test rig that represents the operating context for the software under test as close as possible to physical devices. Two ways to execute automated test: 1. Run well-set pipeline and mark test case result automatically 2. Run commands on Raspberry Pi, manually check and mark the result	QA	Each QA branch release Each Prod branch release Start from POC
Full function test	Full function test will be performed all test suit to check all feature meet the requirement or not.	QA	Each Prod branch release Start from POC
App integration test	App integration test will use engineering App to check App-related features and make sure device firmware can be well coordinated with App software design.	QA	Each QA branch (optional) Each Prod branch release Start from POA
Stability test	Stability test is a method used to evaluate how software performs under certain conditions. The standard requirement pls refer FW Stability Test Guideline V1.0.pdf	QA	Each QA branch (optional) Each Prod branch release Start from POA

Commented [WX34]: Updated the activity description and frequency, clarified the beginning point, since automated test will be routine test for upcoming new E2E project.

Commented [WX35]: Clarified the beginning point here.

Commented [WX36]: Added a new type, because using engineering App to test will become routine for upcoming new E2E project.

Commented [WX37]: Clarified the beginning point. Please note that the new stability test standard document is being drafted and will be updated here later.

Test Type	Test Activity	Ownership	Frequency
Subject test	Subject testing is crucial for identifying and resolving potential problems early in the development process. Examples of features that might undergo subject testing include but not limited to: 1. Fuel gauge user level acceptance test 2. Pod liquid accuracy user level acceptance test	QA	Based on need

15.4 Software test plan management

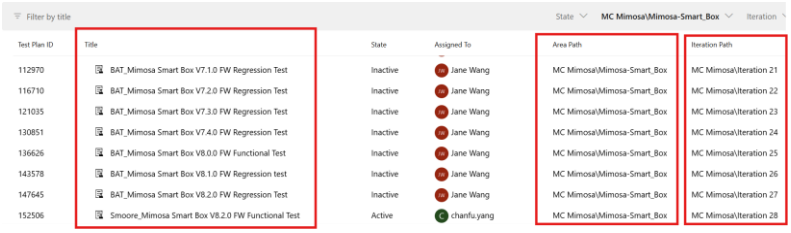
15.4.1 Software test plan

I. Software test plan creation

Each test for QA branch release or Main branch release, there should be a test plan for it to record the test result in ADO. Test plan should come with standard-format title, correct Area Path and incremental Iteration Path.

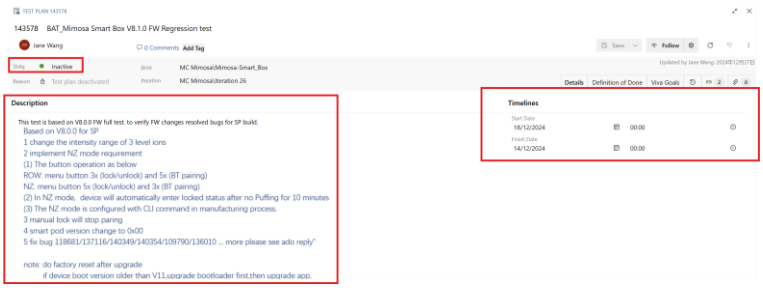
For test plan title naming rule:
Test plan initial owner_Project Name Vx.x.x FW Test Type.

For example:
BAT_Mimosa Smart Box V7.1.0 FW regression test
BAT_Mimosa Smart Box V7.1.0 FW stability test
BAT_Mimosa Smart Box V7.1.0 FW fuel gauge test
BAT_Mimosa Smart Box V8.0.0 FW full function test



II. Software test plan configuration

After creating test plan, should describe more testing details through More Actions > Edit test plan.
State: To reflect test plan state. State should be Active when plan is running and should be Inactive when test is finished..
Description: To describe detailed information about this test plan. Especially for some milestone FW versions.
Timelines: The start Date and Finish Date for the test plan.



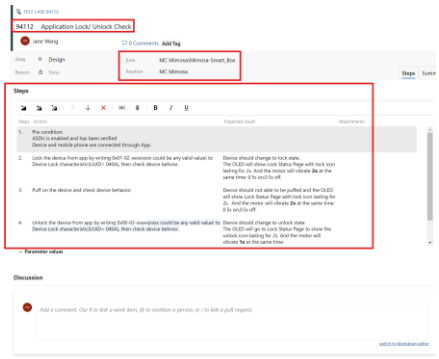
15.4.2 Software test case

I. Software test case creation

For Regression test or Full function test:

Test cases should consist of clear Title, correct Area, Iteration and Steps. Once the case is saved, system will grant this case an unique Bug ID. Steps should comprise below factors as well.

- Pre-condition: to clearly define the device status for this case,
- Actions: to describe operation steps on the device,
- Expected Result: to check if device current behaviour meet the requirement.

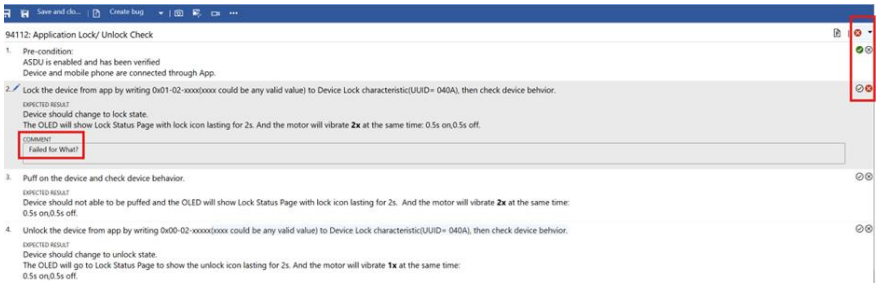


For Stability test or subject test.

- Don't requires the standard test case by using ADO template.
- Upload the test case which design by excel as attachment into ADO is acceptable.

II. Software test case execution

When executing test cases, each test case should end up with a clear result(Pass, fail, pause, block or NA), if not pass, should comment the reason.



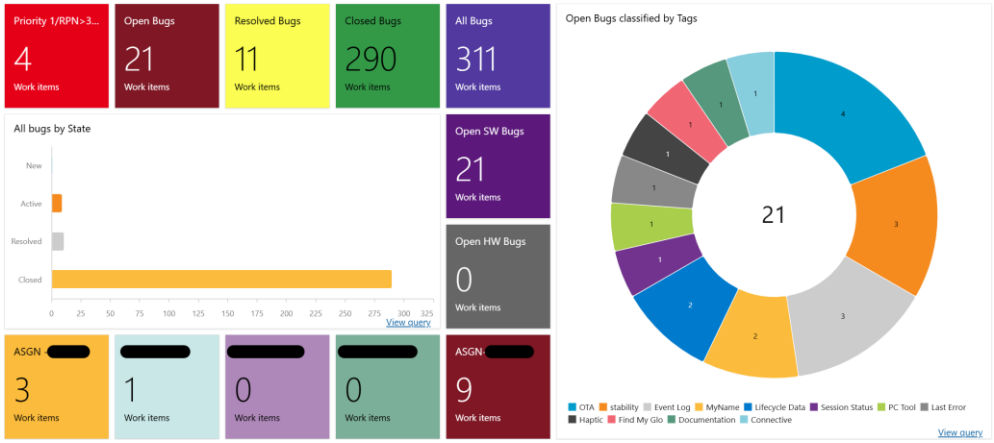
15.5 Software test bug management

15.5.1 Software test bug overview

BAT Test Manager plays a vital role in enhancing the overall quality and reliability of the software, test manager should involves coordinating all development and test resources to monitor the test bugs state.
BAT test manager should use ADO to create the test bugs dashboard for overview.

ADO test bugs dashboard should include below necessary widget:

- Priority 1/ RPN>32 bug state
- Open bugs state
- Resolved bugs state
- Closed bugs state
- All bugs state
- Bugs assign to each responsible
- Bugs classified detail



15.5.2 Software test bug assessment

The severity of bugs is assessed by fields, Priority and RPN. The bug creator should refer to the following rules and determine the Priority and the RPN of the bug.

I. Test bug priority

- Priority 1: should be fixed in advance and bug creator should inform the management team.
- Priority 2: should be fixed ASAP.
- Priority 3: must be fixed before DP, but the fix is not urgent.
- Priority 4: can be fixed after DP.

II. Test bug RPN （Risk priority number）

Risk Priority Number (RPN) is a scientifically calculated method used to assess the impact of bugs. The RPN is determined by multiplying three factors: Impact, Occurrence, and Detection. The calculation formula is

PRN = Impact * Occurrence * Detection

For each factors detail value as below:

- Impact:
  - Impact 5: safety issue and launch blocker.
  - Impact 4: severe issue, major function lost, not recoverable, cause high user dissatisfaction.
  - Impact 3: severe issue, major function lost, recoverable, cause high user dissatisfaction.
  - Impact 2: minor issue.
  - Impact 1: no user impact.
- Occurrence:
  - Occurrence 5: repro rate >= 10%
  - Occurrence 4: 1% <= repro rate < 10%
  - Occurrence 3: 0.1% <= repro rate < 1%
  - Occurrence 2: 0.01% <= repro rate < 0.1%
  - Occurrence 1: repro rate <= 0.1%
- Detection:
  - Detection 5: use scenario for marketing users.
  - Detection 4: frequent use scenario.
  - Detection 3: average use scenario.
  - Detection 2: rare use scenario.
  - Detection 1: corner case.

Note: Bugs with Priority of 1 and RPN of higher than 32 should be highlighted.

15.5.3 Software test bug lifecycle

The bug lifecycle, also known as the defect lifecycle, is a series of stages that a bug goes through from its identification to its resolution. This process helps in tracking and managing bugs efficiently, ensuring that they are addressed systematically.

There are six states of a bug in ADO.

- New: Assigned to states associated with newly added work items so that they appear on the backlog.
- Active: Assigned to states that represent active work.
- Resolved: Assigned to states that represent a solution was implemented but not yet verified.
- Closed: Assigned to states that represent work that's complete.
- Inactive: Assigned to states that represent the limitation of this generation of the product, which means the bug will not be fixed but is recorded and might be fixed in the next generation.
- Monitoring: Assigned to states that represent long-time monitoring of the resolved bug.

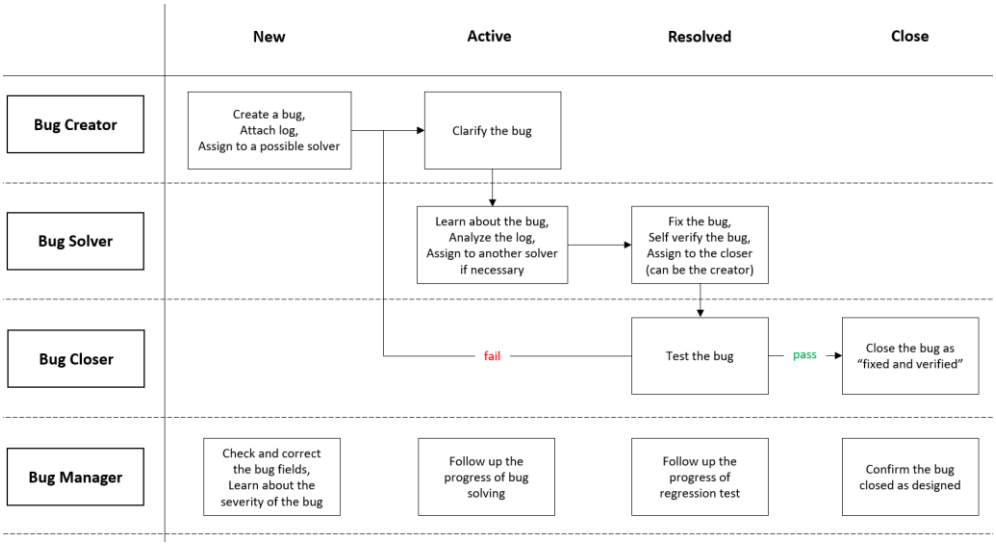
Commented [WX38]: Add two states for E2E project.

There are five scenarios during bug lifecycle:

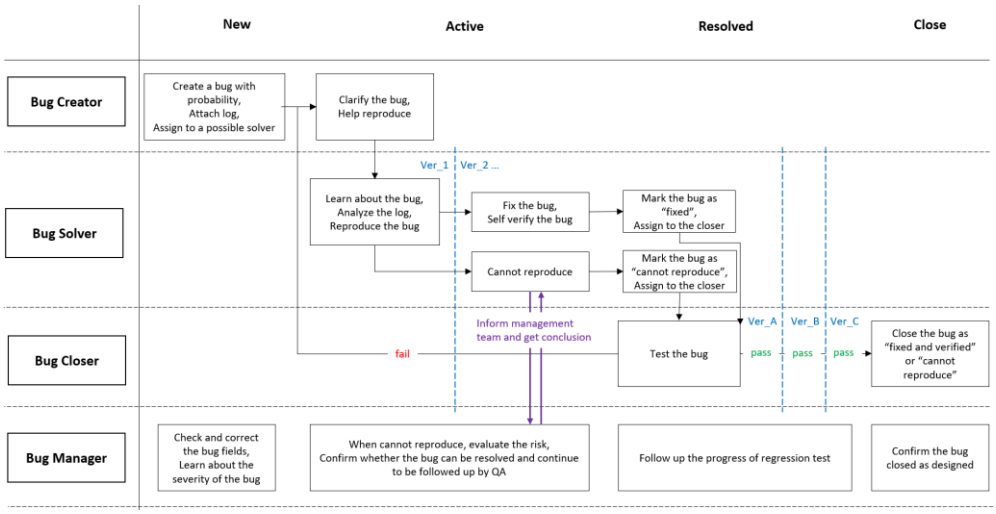
- Fixed and Verified: bug is clear and can be fixed.
- Bug with Probability / Cannot Reproduce: bug is with a probability, might not reproduce any more.
- Will Not Fix: there is controversy over whether to modify, if not modified, need conclusion to confirm.
- As Designed: device acts as designed, tester might misunderstand.
- Duplicate: duplicate of a previous bug.

Pls find each scenarios work flow:

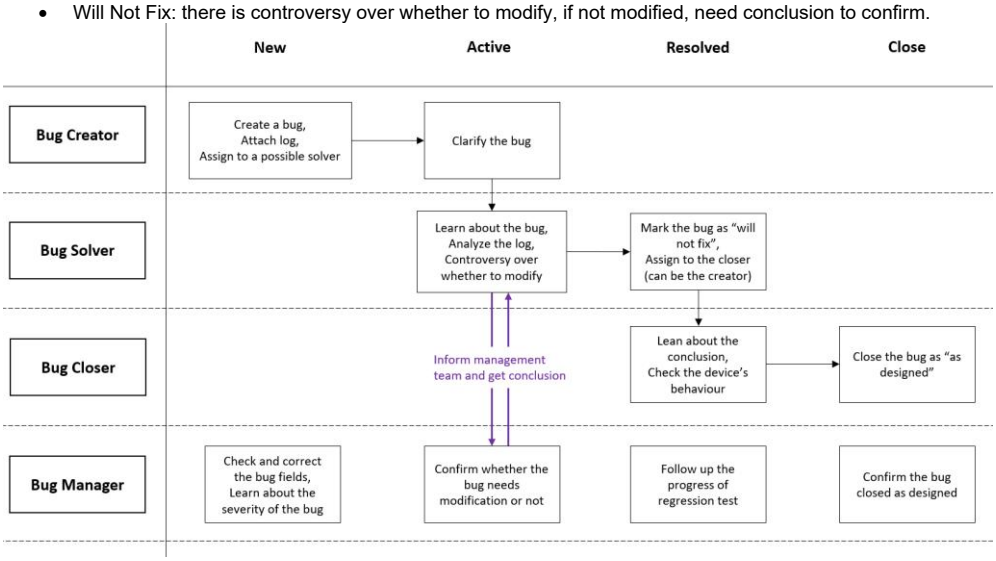
- Fixed and Verified: bug is clear and can be fixed.



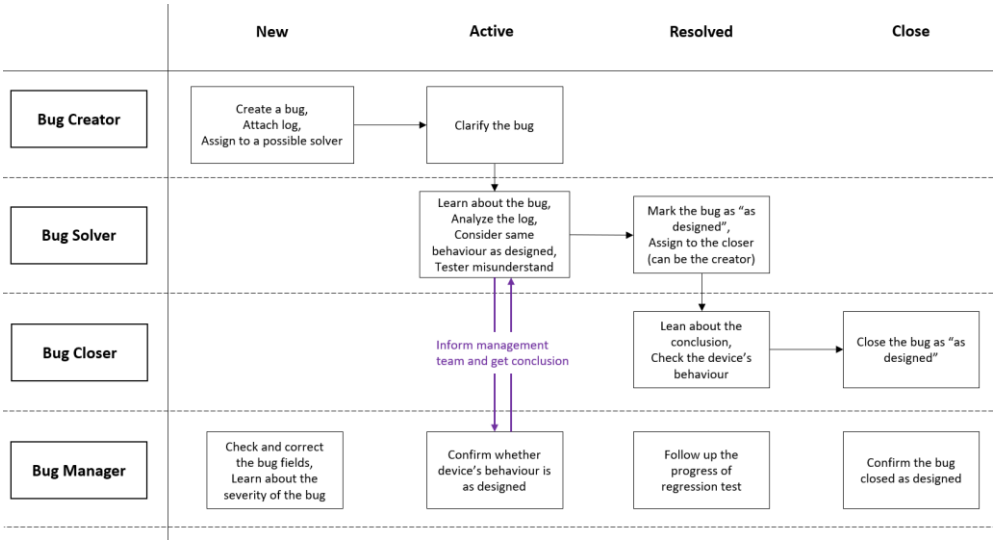
- Bug with Probability / Cannot Reproduce: bug is with a probability, might not reproduce any more.



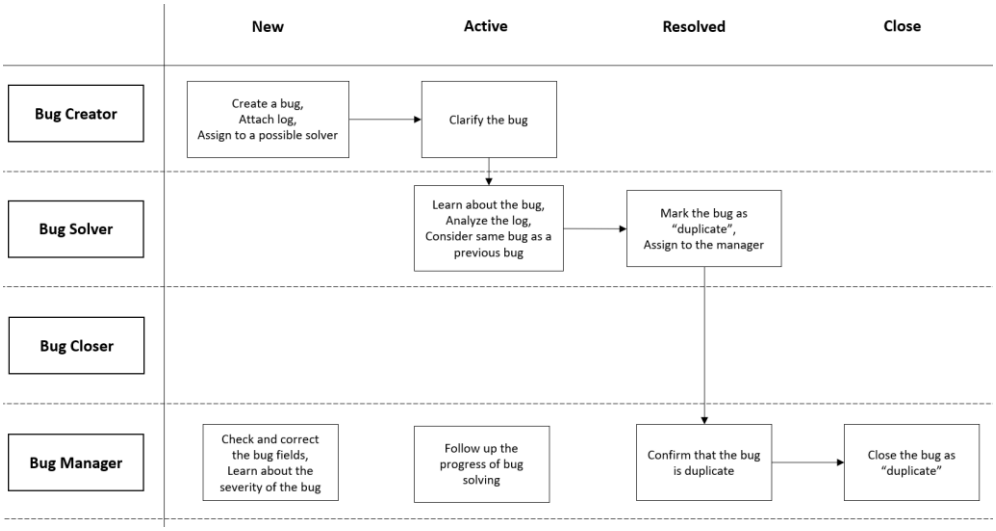
- Will Not Fix: there is controversy over whether to modify, if not modified, need conclusion to confirm.



- As Designed: device acts as designed, tester might misunderstand.

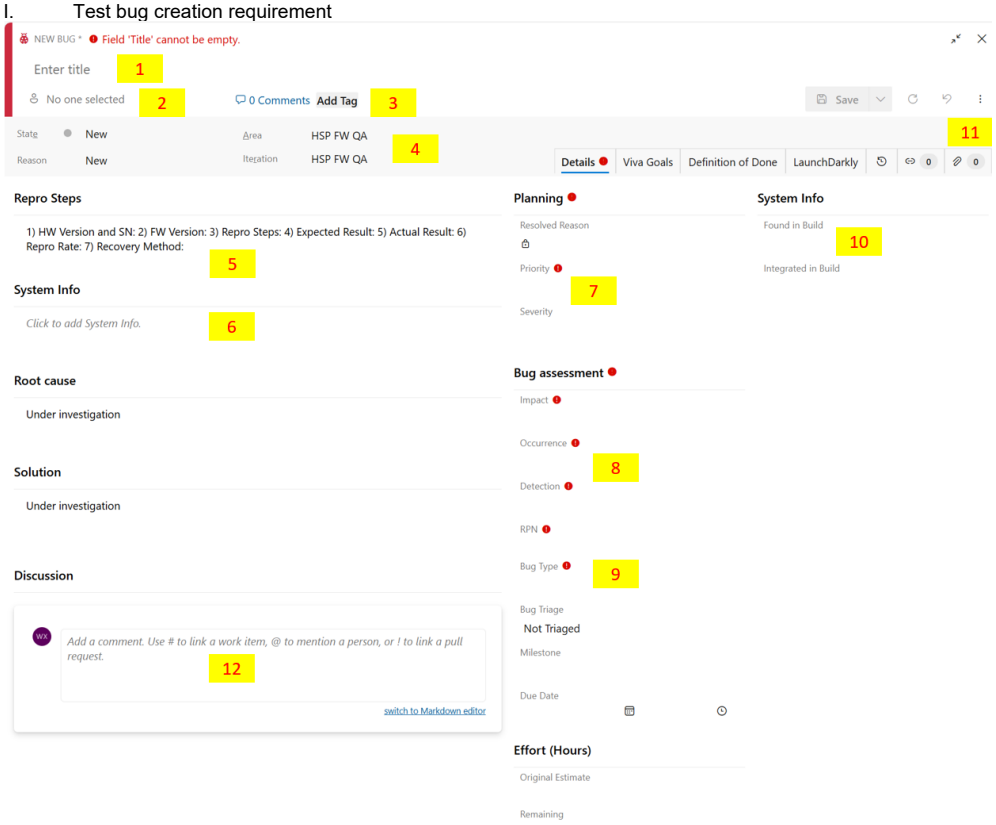


- Duplicate: duplicate of a previous bug.



15.5.4 Software test bug creation/resolve requirement

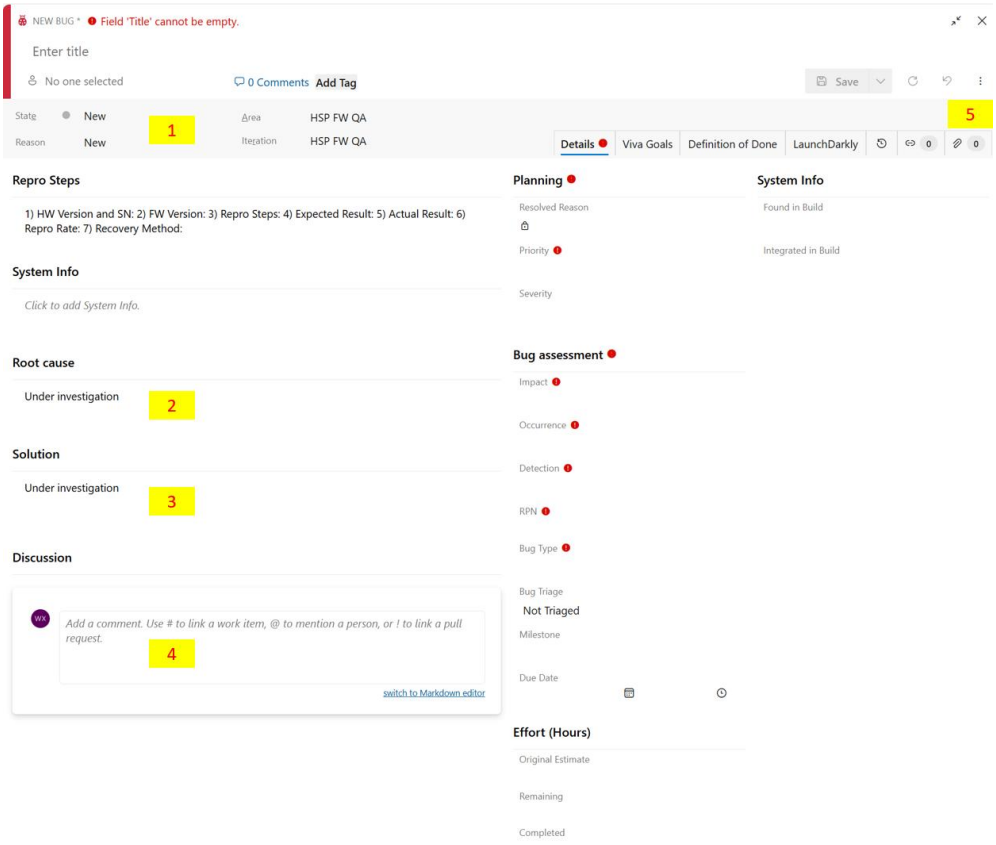
I. Test bug creation requirement



Create a bug directly in the project or create a bug when executing a test case.

- Step 01: Title: a concise and clear sentence to describe the problem, summarize the steps and device's behaviours.
- Step 02: Assigned to: select the possible bug solver.
- Step 03: Tags: add a tag in the tag list to categorize the bug, tag list of the project should be defined by the management team.
- Step 04: Area and Iteration: fill in the area path and iteration path according to the requirement of the project.
- Step 05: Repro Steps: fill in the information as below.
- Step 06: System Info: HW version, FW version, SN, and all other information of the device system.
- Step 07: Priority: fill in the priority of the bug, refer to 15.5.1 Bug assessment.
- Step 08: RPN: fill in the impact, occurrence, and detection of the bug, calculate the RPN, refer to 15.5.1 Bug assessment.
- Step 09: Bug Type: select the bug type.
- Step 10: Found in Build: the FW version in which the bug is found.
- Step 11: Attachment: log, screenshots, videos, and all other relevant information that can help the solver understand and fix the bug.
- Step 12: Comment: anything that needs to be highlighted or clarified, comment here.

II. Test bug resolve requirement



For resolve bug, steps as below:

- Step 1: State and Reason:
  - Active: start to work on the bug, state should be set to “Active” and no need to change reason.
  - Resolved: bug already fixed or waiting to be closed because of other reasons, state should be set to “Resolved” and reason should correspond to the five situations above, including: fixed and verified, cannot reproduce, will not fix, as designed, duplicate.
- Step 2: Root cause: the cause of the bug, need to be clear, cannot just say that something wrong.
- Step 3: Solution: describe how the code is modified clearly, or explain why no modification, cannot just say that code changed.
- Step 4: Comment: anything that needs to be highlighted or clarified or determined, comment here.
- Step 5: Attachment: log, screenshots, videos, temporary firmware, and all other relevant information that can help fix the bug.

15.6 Software test report management

Test completion reporting is a process, whereby the test metrics are reported in summarised format to update the stakeholders. This enables them to take an informed decision.

Test report contains the following information:

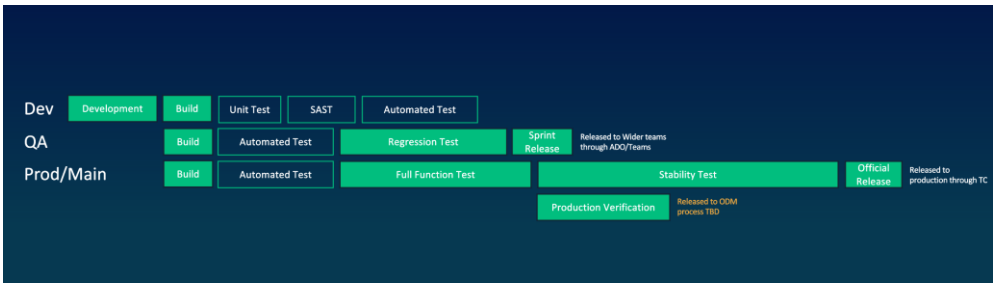
- Tester
- Test date
- Test summary
- Test result
- Test bug information

For each QA branch release version, test report could be simple by using the excel format report template, not require upload to Teamcenter.

For Main branch release version, test report should use the word format report template, require upload to Teamcenter.

16 FW PRODUCTION RELEASE PROCESS

The diagram illustrates a firmware (FW) release process across three stages: Dev, QA, and Prod/Main. Below is a definition of each block in the process:



16.1 Development Phase

16.1.1 Development

Initial firmware development by FW Engineers.

16.1.2 Build

Code is compiled into executable firmware. A build is not a release but it is a fundamental part of the release process, without a successful build there shouldn't be a release. BAT has a build pipeline that is triggered at a number of different points, when the build is triggered it runs through a number of stages which are highlighted below.

Build Firmware Images

When the build is initially triggered, it will need to take the source code and compile it into a number of binaries that can be used to "Flash" the device. The list below highlights what these may be. This isn't an exhaustive list and can be updated:

- **Bootloader (.bin .elf .hex formats)**
- **Application (.bn .elf . hex formats)**
- **Memory Map**
- **OTA packages**

16.1.3 Unit Test

Basic testing of individual components or functions. (TBD). The next stage in the build process is to perform some unit tests which will test the individual functions of the firmware. The purpose is to test the smallest unit of the firmware rather than all of the firmware operations at the same time. By doing this, we are simply testing that the smallest unit of the firmware behaves as expected. A unit test should be written for every function as the firmware is developed. When the build is run, all of the combined unit tests should run and give a result, these results should be collated into a report, which in turn can be published as an artefact for QA purposes later.

16.1.4 SAST (Static Application Security Testing)

Code is scanned for security vulnerabilities and coding issues without executing the code. Static Analysis Security Testing is not currently a stage in the pipeline but one that BAT are intending to implement to improve the security posture of the devices that are developed. It is expected that when this is implemented, it will run and create a report that can also be included as an artefact in the release.

Commented [AT39]: Suggest that we add SonarQube scan in this section:

To further strengthen code quality and security during the development phase, automated static code analysis will be integrated into the pull request process. This allows potential code quality, maintainability, and security issues to be identified earlier before changes are merged into the target branch. SonarQube will be used as the current static code analysis tool for this process.
SonarQube scanning will be triggered automatically whenever a pull request is created. At the POA stage, the SonarQube quality gate will be enabled to block any pull request containing High or Blocker issues. Developers must resolve these issues before the pull request can be completed. Starting from the POC stage, the quality gate will also block Medium-level issues. Developers can also access SonarQube during code development to review issues earlier and avoid unexpected findings when creating pull requests. In urgent cases, the project PM may request the DevOps team to bypass the quality gate, with appropriate justification and approval recorded.
For more details on the SonarQube process, developers can reference the Wiki page: https://dev.azure.com/ado-bat-uksouth-prod-01/FW_Platform/_wiki/wikis/FW_Platform.wiki/1721/SonarQube-%F0%9F%94%8D
In addition, an AI-based code scanning capability is currently under development and may be introduced into the SDLC process once it is validated and ready for adoption. This future capability is intended to further support early issue detection and improve code quality, but it is not yet part of the mandatory quality gate criteria.

@Dennis Du @Oliver Zheng (External) Please help check if modification is required.

16.1.5 Dev Release

This is software that is in an alpha stage. The release can be created at any point from the development branch.

16.2 QA Phase

16.2.1 Build

Firmware is built again to ensure environment consistency.

16.2.2 Manual & Automated Functional Test

QA-level automated tests to validate basic functionalities. The functional or system testing is the installed binary from the "Build Firmware Images" stage, running directly on the target hardware. In the BAT pipeline this is controlled through the Robot Framework, which is internally known as Medusa. The device is controlled via a Command Line Interface, which allows for a level of automation to check the device is behaving as it should when certain tasks such as Charging, Puffing and Bluetooth Connectivity are performed. This stage creates a test report, which is an artefact that should be included in a release.

16.2.3 Regression Test

Ensures new changes do not break existing functionality.

16.2.4 Sprint Release

A release at the end of a sprint; shared with wider teams via Azure DevOps (ADO)/Teams.

OR

16.2.5 QA Release This is software that is in a Beta status and should be tested internally by BAT.

16.3 Production/Main Release Phase

16.3.1 Build

Production-level build of the firmware.

16.3.2 Automated Test

Final round of automated testing to validate stability.

16.3.3 Full Function Test

Comprehensive manual or automated test covering all scenarios.

16.3.4 Stability Test

Stress testing to verify firmware reliability and performance.

16.3.5 Production Verification

Tests performed to verify that firmware is ready for deployment

16.3.6 **PRD release** This is software that is ready to be released to the end consumer and flashed onto production devices.